SentinelOps SOC Alert Triage Debugging and Optimisation Report

Student: Hitesh C Register number: 192524005 Course: CSA06 - Design and Analysis of Algorithms Assessment: CO2 Debugging and Optimisation Task

This report records the defects in the supplied alert-triage code, the minimal corrections, the optimized algorithms, test results, complexity analysis, and measured comparisons. The original, corrected, and optimized Python programs are included in `Source_Code/`.

Debugging, optimisation, analysis, and delivery

Detailed findings

Part A: Reproduce and identify

The original functions were copied into `Source_Code/original_module.py`. `Source_Code/reproduce_original.py` runs them against the five data values in the question. The expected outputs below follow the stated behavior: stable descending severity, True when an IP is blocked, every exact signature position, and the minimum Euclidean distance.

Function	Expected output	Actual output from supplied code	Main symptom
`sort_alerts`	`A2(9), A4(9), A3(5), A1(3), A5(1)`	`A4(9), A3(5), A3(5), A5(1), A5(1)`	First record is lost and records are duplicated.
`is_blocked`	`[True, True, False]`	`[False, False, False]`	Hits are reported as not blocked; search stops at the first list item.
`find_signature`	`[0, 8]`	`[0, 4]`	Final real match is omitted and a partial match is reported.
`closest_logins`	Pair `(0, 5)`, distance `1.4142 km`	Pair `(0, 2)`, distance `60.4401 km`	The function selects a far-apart pair.

Nine defects in the supplied source

Line numbers below refer to the copy in `Source_Code/original_module.py`.

ID	Function and line	Defect	Root cause and effect
D1	`sort_alerts`, line 8	`j > 0` excludes index 0.	The first alert is never compared or shifted, so a new key cannot move to the front.
D2	`sort_alerts`, line 11	The key is stored at `alerts[j]`, not `alerts[j + 1]`.	It overwrites an existing alert after shifts, causing data loss and duplicates.
D3	`is_blocked`, line 18	A matching address returns `False`.	A confirmed blocklist hit is classified as allowed.
D4	`is_blocked`, lines 19-20	`else: return False` is inside the loop.	A miss against the first entry ends the search before later entries are checked.
D5	`find_signature`, line 26	`range(n - m)` omits the final valid start.	A match beginning at `n - m` is never tested.
D6	`find_signature`, line 30	`j == m - 1` is used as proof of a complete match.	If the last character mismatches, `break` leaves `j` at `m - 1`, causing a false hit. A true match is also ambiguous without tracking loop completion.
D7	`closest_logins`, lines 36 and 42	Starts at zero and updates when `d > min_dist`.	It keeps the largest distance, not the smallest.
D8	`closest_logins`, lines 38-39	Both loops include all indexes.	It computes self-distances and repeats every distinct pair in both orders.
D9	`closest_logins`, line 40	A square root is computed for every candidate.	Square root is unnecessary for ordering distances; it adds work to every pair.

Part B: Correction order and verification

Each correction was made in `Source_Code/corrected_module.py` while retaining the same algorithmic approach. The harness reruns each function after repair and checks boundary cases.

`Source_Code/stepwise_debug.py` rebinds one corrected function at a time into the original module and reruns the four sample checks after each function repair. Its saved transcript shows the repaired function switching to PASS while the still-unfixed functions remain FAIL; after the fourth repair all four pass.

Sorting trace

The corrected insertion sort moves an alert only while the previous severity is strictly lower. The two defects are repaired independently: allow `j` to reach -1, then insert the saved key at `j + 1`.

Pass	Key	Relevant comparisons and shifts	Corrected list after insertion
Start	-	-	`A1(3), A2(9), A3(5), A4(9), A5(1)`
1	`A2(9)`	`A1(3) < 9`; shift A1 right; insert at index 0.	`A2(9), A1(3), A3(5), A4(9), A5(1)`
2	`A3(5)`	Shift A1(3); stop at A2(9); insert at index 1.	`A2(9), A3(5), A1(3), A4(9), A5(1)`
3	`A4(9)`	Shift A1(3), A3(5); stop at equal A2(9); insert after A2.	`A2(9), A4(9), A3(5), A1(3), A5(1)`
4	`A5(1)`	Compare with A1(3); no shift; restore key at index 4.	`A2(9), A4(9), A3(5), A1(3), A5(1)`

The strict comparison preserves the input order of A2 before A4, which both have severity 9.

Blocklist trace

The loop now returns True only when the current address equals the query. It continues after a miss and returns False only after exhausting the list.

Query	Search path	Expected and corrected result
`10.0.0.9`	Compare `.5` (miss), then `.9` (hit).	`True`
`10.0.0.5`	First entry hits.	`True`
`8.8.8.8`	All three entries miss.	`False`
any address, empty list	No comparisons; final return executes.	`False`

Signature alignment trace

For `log = FAILFAIXFAIL` and `pattern = FAIL`, valid alignment starts are 0 through 8. The original loop tested only 0 through 7, and the `j` test mistook a mismatch on the final pattern character for a full match.

Start	Compared text	Result
0	`FAIL`	Full match
1	`AILF`	Mismatch at first character
2	`ILFA`	Mismatch at first character
3	`LFAI`	Mismatch at first character
4	`FAIX`	Mismatch at final character; original incorrectly reports 4
5	`AIXF`	Mismatch at first character
6	`IXFA`	Mismatch at first character
7	`XFAI`	Mismatch at first character
8	`FAIL`	Full match; original loop does not visit this start

The corrected brute-force version includes start `n - m`, sets a match flag, and tests `pattern longer than log`, an empty pattern, and overlapping matches.

Closest-pair correction

The minimal correction returns `(None, infinity)` for fewer than two events, skips `i == j`, starts with infinity, and updates on a smaller distance. It retains the all-ordered-pairs scan and a square root for each non-self pair so that the later optimization can be measured separately. Ties are resolved by the lexicographically earliest pair of indexes.

Test	Expected	Corrected output
Supplied six events	Pair `(0, 5)`, distance `sqrt(2)`	Pair `(0, 5)`, distance `1.414213562...`
Empty events	No pair, infinity	`(None, inf)`
One event	No pair, infinity	`(None, inf)`
Two points `(0,0)` and `(3,4)`	Pair `(0,1)`, distance 5	Pair `(0,1)`, distance 5

Part C: Optimization choices

Function	Corrected approach	Optimized approach	Reason
Alerts	Stable insertion sort	Stable Python Timsort	Insertion sort is linear on already sorted input but quadratic when many records are out of order. The workload can have 50,000 alerts; Timsort is O(n log n) worst-case and adaptive to existing runs.
Blocklist	Sequential scan	Build one `frozenset`, then reuse membership checks	Set construction is O(m); each lookup is O(1) average rather than O(m), useful across many queries.
Signature	Brute-force alignments	KMP prefix table and scan	KMP uses previously matched prefix data and runs in O(n + m), including overlapping occurrences.
Closest pair	Corrected all-pairs scan	Divide-and-conquer by x, y-sorted strip pruning	Compares only candidate cross-half pairs within the best known x/y bounds, avoids self and repeated pairs, compares squared distances, and takes one square root at the end.

Part D: Complexity and measurements

Complexity

Function	Corrected time	Optimized time	Auxiliary space
Sort n alerts	Best O(n), average/worst O(n^2)	O(n log n) worst-case; often O(n) on ordered runs	O(1) insertion; O(n) Timsort
Search m blocked IPs	O(m) per query	O(m) build, O(1) average per query	O(1) scan; O(m) set
Match length-m pattern in length-n log	O((n-m+1)m) worst-case	O(n + m)	O(k) results; O(m) LPS table
Closest pair of n events	O(n^2), and corrected version evaluates n(n-1) ordered non-self pairs	O(n log n) worst-case	O(n) auxiliary data and recursion

The search set's worst-case lookup can degrade under pathological hash collisions. Timsort's input-dependent best case does not change its O(n log n) worst-case guarantee. The KMP comparison counter includes only comparisons between a log character and a pattern character.

Closest-pair experiment

The PDF's sentence ends after “between”; the two endpoints are missing in the supplied document. The production scenario names 8,000 events, so the requested interval is interpreted as 1,000 to 8,000. Measurements use 1,000, 4,000 and 8,000 events from one deterministic random dataset (seed 20261026). Each timing is one run on the Codex workspace, not a universal performance claim. A “distance evaluation” means a full candidate squared Euclidean distance; strip-bound checks are not counted as full distances.

Events	Brute-force pair distances	Divide-and-conquer pair distances	Reduction	Brute ms	Optimized ms
1,000	499,500	923	99.8152%	81.197	4.333
4,000	7,998,000	3,765	99.9529%	1,339.964	19.242
8,000	31,996,000	7,451	99.9767%	10,456.123	96.427

The measured distance counts agree with the analysis: brute force grows quadratically, while the divide-and-conquer method uses far fewer candidate distances for this distribution. Its sorting and recursive partitioning still take O(n log n) time in the worst case.

Insertion-sort experiment

Both experiments used 5,000 alerts. The nearly sorted sequence started in descending severity and had 100 adjacent inversions. The random sequence used a fixed seeded shuffle. Counts include severity comparisons only; times are one run.

Input	Insertion severity comparisons	Insertion ms	Timsort ms	Same result
Nearly sorted	5,099	1.550	0.658	Yes
Random	6,292,612	1,157.879	1.678	Yes

The nearly sorted result shows why insertion sort can be useful for small or almost ordered inputs. The random result shows why it is not suitable as the only production sort for 50,000 alerts.

Part E: Verification and delivery

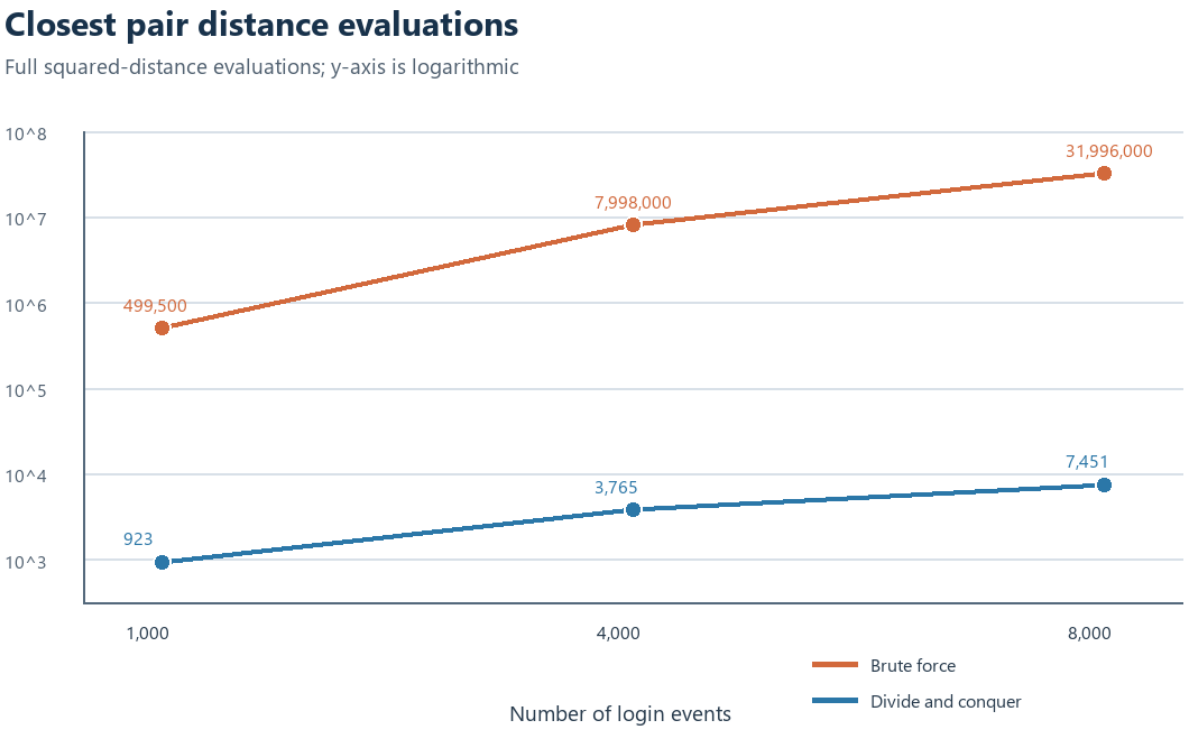
`Source_Code/test_harness.py` ran five test groups successfully. It checks the sample results, stable ties, empty blocklist, signature at the final valid position, longer-than-log pattern, empty pattern, overlaps, fewer than two events, duplicate coordinates, ties, and randomized equivalence tests against corrected baselines. It includes 250 randomized closest-pair inputs, 500 randomized string inputs, and 100 randomized sorting inputs.

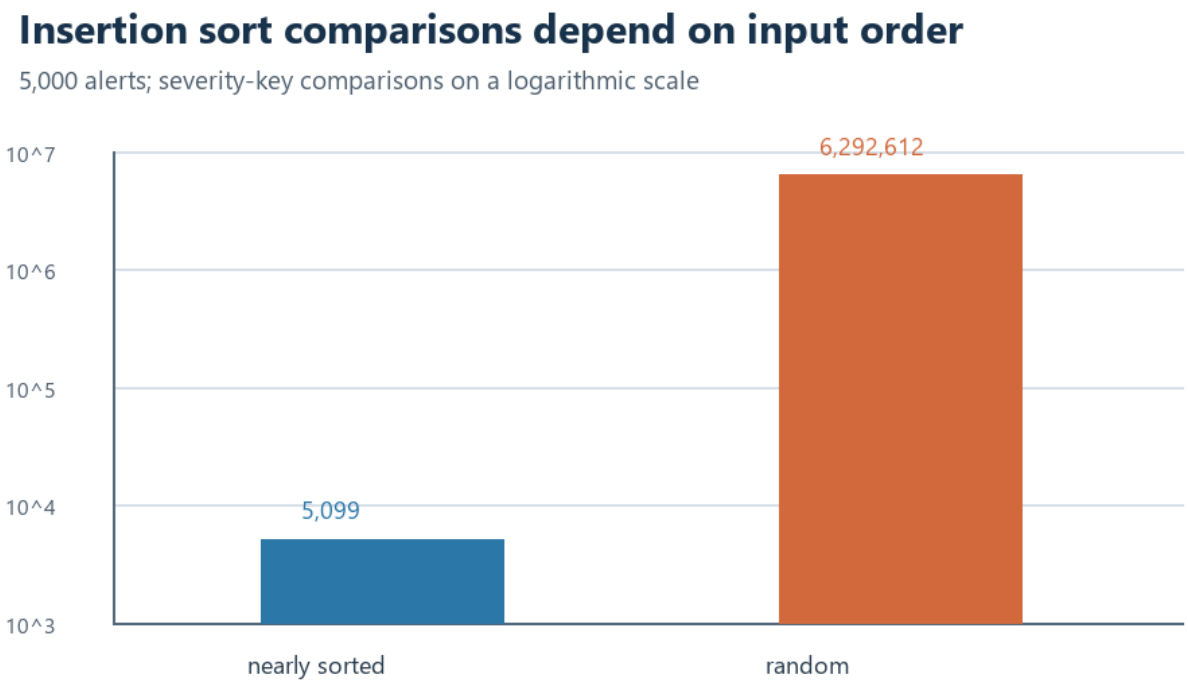
`Supporting_Files/distance_experiments.csv`, `sort_experiments.csv`, and the execution log preserve the measured results. The test dataset and charts are also included. Build and run commands are in the top-level README.

The code and report were prepared with AI assistance. Hitesh should review the work, follow course rules on assistance, and be ready to explain all corrections and complexity claims in class. The attached brief says the work is individual and requests a repository link through Google Classroom; submit the linked repository folder only, as directed.

Appendix A: Experimental charts

Closest-pair pair-distance evaluations

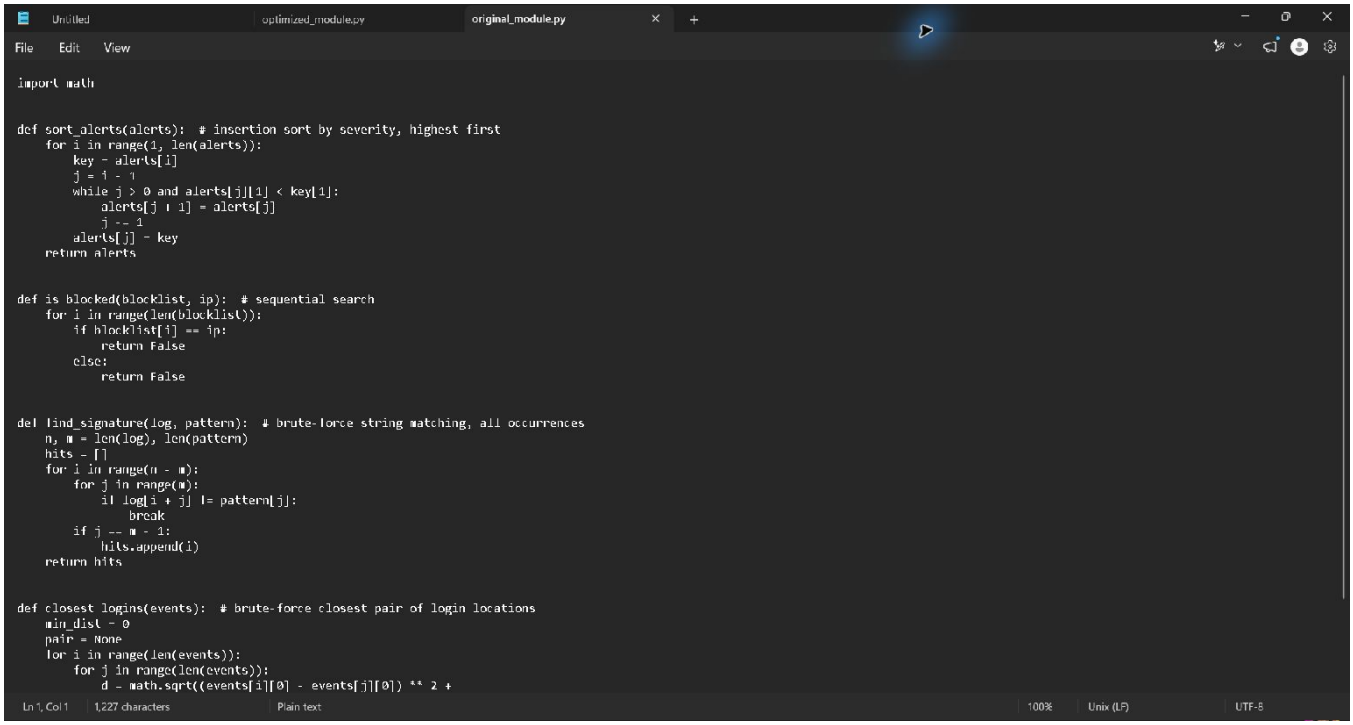



Insertion sort comparisons depend on input order
5,000 alerts; severity-key comparisons on a logarithmic scale
10^7
10^6
10^5
10^4
10^3
6,292,612
5,099
nearly sorted
random
Nearly sorted input needs far fewer comparisons than shuffled input.

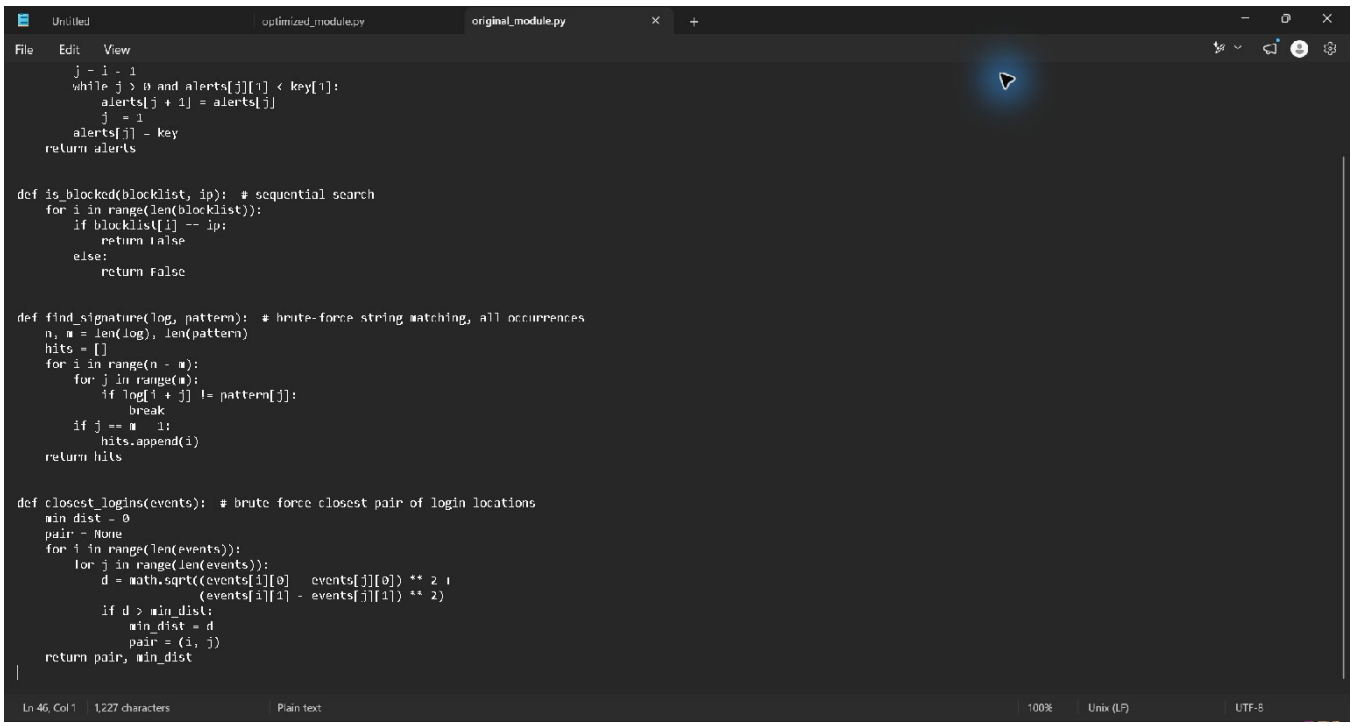
Appendix B: Source and output evidence captures

These are captures of source files and saved run logs displayed in Notepad. The complete source and text logs are included in the task folders. The output logs came from actual Python runs in this workspace; the images are not live terminal screenshots.

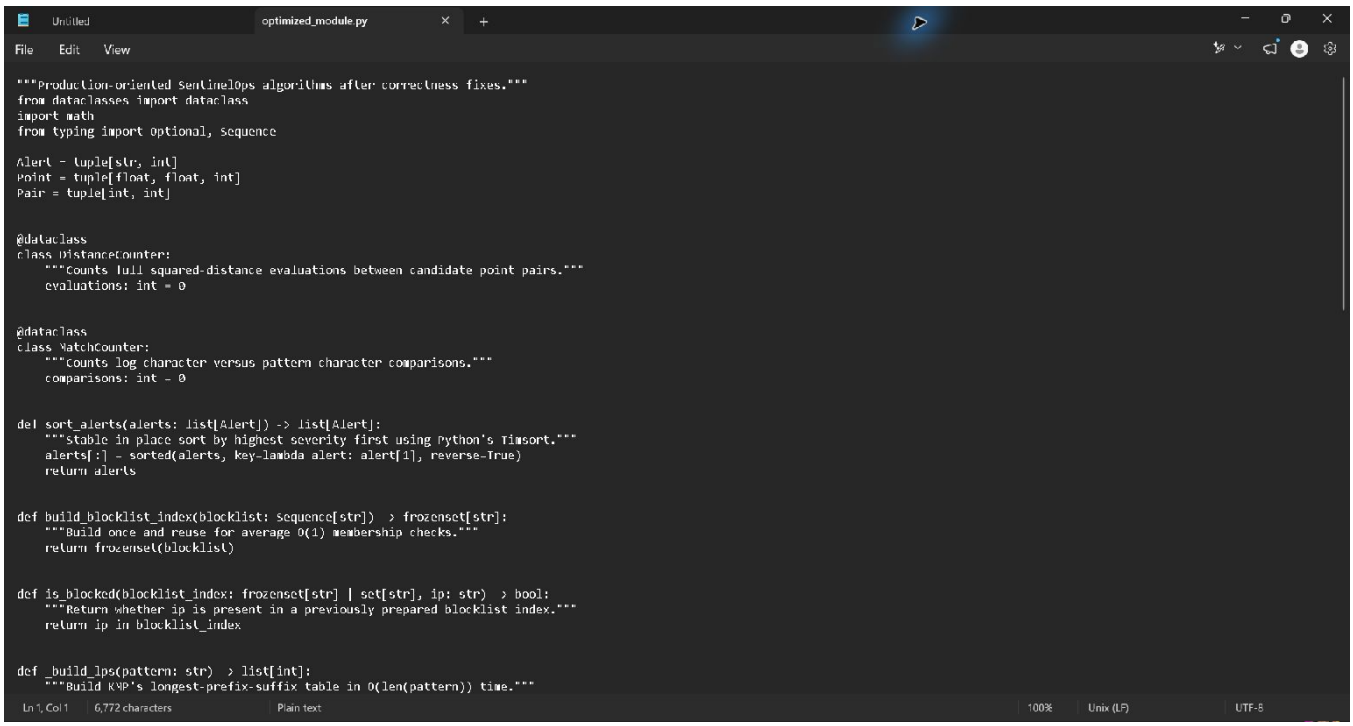
Original source code, first section



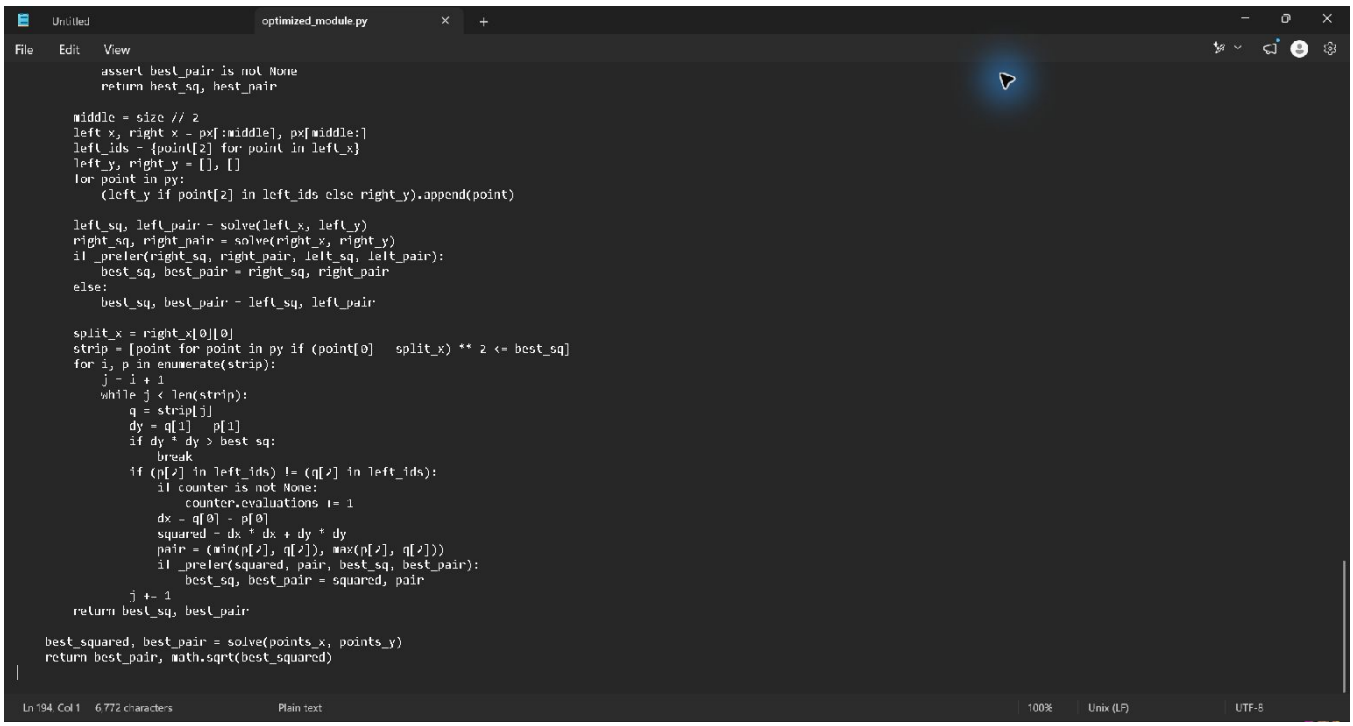
Original source code, closest-pair section



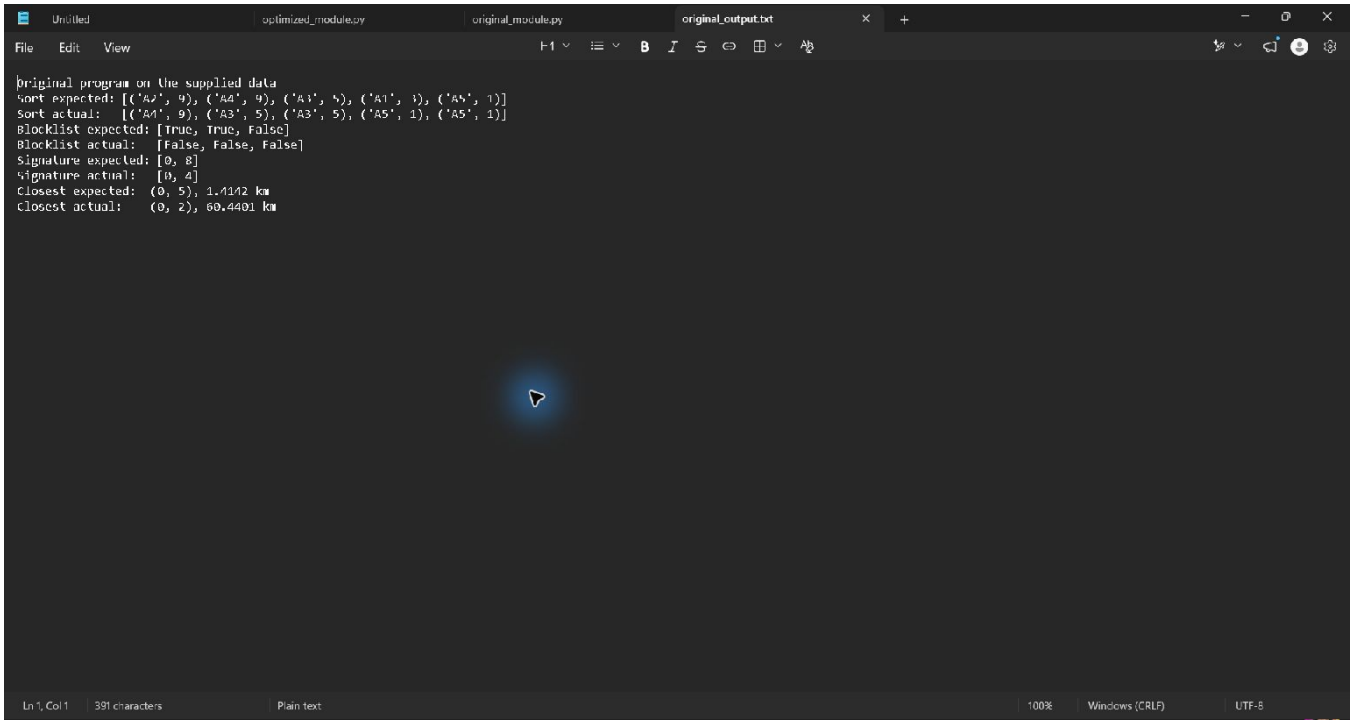
Optimized implementation, first section



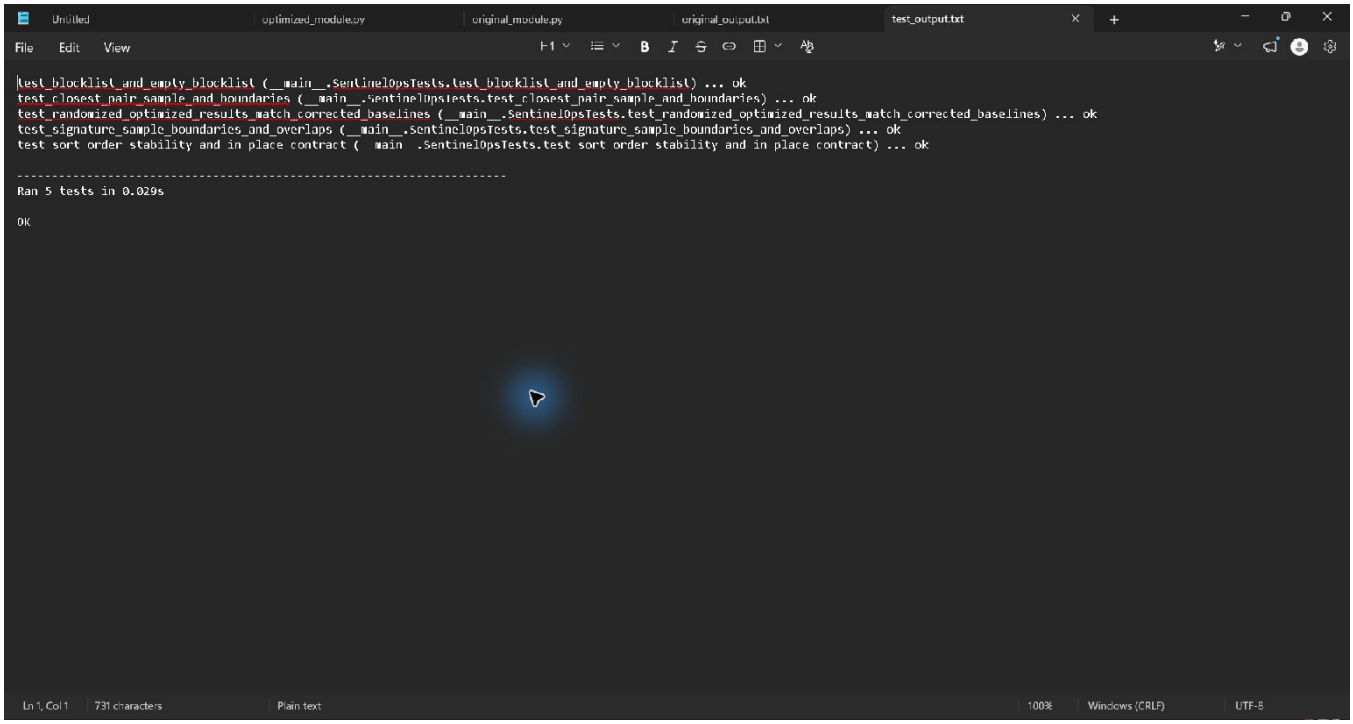
Optimized closest-pair implementation



Original program output



Corrected and optimized test output



Experiment output

